DEEP LEARNING · CNN ARCHITECTURES · STUDY NOTES

LeNet-5, Classic and Modern

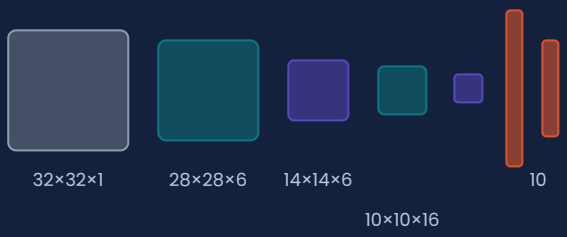
The 1998 network layer by layer, the modern version, and a hand calculation of everything that changed. Every slide from *LeNet5_Classic_and_Modern.pptx*, explained from scratch.

LeCun, Bottou, Bengio & Haffner · 1998

60,000 parameters

340,908 connections

0.95% MNIST error



00 · Foundations
The four formulas, and parameters vs connections.

01 · The big picture
History (1989 → 2012) and the original network end to end.

02 · Part A: original 1998 LeNet-5
C1, S2, sparse C3, S4, C5, F6, RBF output, and the full tally.

03 · Key insight
Weight sharing: each parameter is reused up to 784 times.

04 · Part B: modern LeNet-5
Every change worked by hand: ReLU, max pooling, full C3, softmax, PyTorch code.

05 · Wrap-up
Takeaways, the CIFAR-10 exercise with answers, and self-test.

READ THIS FIRST · SLIDE 3

00 Foundations

Four formulas used throughout, and the one distinction this deck cares about most.

Convolution layer (C)
A small filter (here always 5×5) slides over the input; at each position it multiplies and adds to make one output value.

Subsampling layer (S)
LeNet's version of pooling: it shrinks each map by summarizing 2×2 windows.

Fully connected layer (F)
Every input connects to every output with its own weight.

Feature map
The output grid of one filter. "28×28×6" means six 28×28 maps.

MNIST
A dataset of 28×28 grayscale images of handwritten digits 0–9. LeNet-5's test set.

Activation (squashing) function
A non-linear function applied after each layer. LeNet uses a scaled tanh; modern versions use ReLU.

The toolkit: four formulas

OUTPUT SIZE

$$O = (W - F + 2P) / S + 1$$

Same formula for convolution and subsampling layers.

CONV PARAMETERS

$$(F \times F \times C_{in} + 1) \times K$$

Weights of each filter plus one bias, times K filters.

CONNECTIONS

$$O \times O \times K \times (F \times F \times C_{in} + 1)$$

Every output value, times the inputs and bias it touches.

FC PARAMETERS

$$(N_{in} + 1) \times N_{out}$$

Every input to every output, plus one bias per output.

W input width/height · **F** filter size · **P** padding · **S** stride · **C_in** input channels · **K** number of filters

PARAMETERS

Distinct numbers to learn. Sets the **memory** the model needs.

CONNECTIONS

The number of multiply-adds, including the bias. This is what the 1998 paper counts. Sets the **compute**.

WHY THEY DIFFER

For a fully connected layer they are **equal**. For a conv layer, **connections = parameters × number of output positions**, because the same filter is reused at every position. That reuse is called **weight sharing**, and it is why connections are far larger than parameters.

SLIDES 2 AND 4 · THE BIG PICTURE

01 The CNN that started it all

1989
LeNet-1
Backprop-trained CNN reads handwritten US zip codes

1998
LeNet-5
The classic paper: 7 layers, about 60K parameters

Late 1990s
In production
Reads an estimated 10% of US bank cheques

2012
AlexNet
Same conv → pool → FC recipe, scaled up with GPUs

0.95%	60,000	340,908
test error on MNIST (0.8% with distortions)	trainable parameters	connections

EVERY INGREDIENT OF A MODERN CNN, ALREADY IN 1998

Local receptive fields (each unit sees a small patch), **shared weights**, **spatial subsampling**, and **end-to-end gradient training**. AlexNet (2012) is essentially LeNet-5 made much bigger, with ReLU, dropout and GPUs. The 10% figure is the commonly cited estimate for the NCR cheque-reading system.

Original LeNet-5, end to end

Input	C1 conv	S2 subsample	C3 conv	S4 subsample	C5 conv	F6 full	Output RBF
32×32×1	28×28×6	14×14×6	10×10×16	5×5×16	1×1×120	84	10

Convolution · Subsampling · Fully connected / output

NAMING AND SHAPE

C = convolution, **S** = subsampling, **F** = fully connected. Height and width shrink **32** → **1** while channels grow **1** → **120**. All conv filters are 5×5 with stride 1 and no padding; all subsampling is 2×2 with stride 2.

TRY IT FIRST

Predict each shape with `0 = (W − F + 2P)/S + 1` before reading the next pages. For C1: (32 − 5 + 0)/1 + 1 = 28.

SLIDES 5–10 · PART A

02 Original LeNet-5 (1998), layer by layer

Follows the paper exactly: trainable subsampling, sparse C3, RBF output.

Step 1 · C1: the first convolution

Input 32×32×1 · K = 6 · F = 5 · S = 1 · P = 0

Output size

`O = (32 − 5 + 0)/1 + 1 = 28`

→ `28×28×6`

Parameters

`(5×5×1 + 1) × 6`

`= 26 × 6 =` `156`

Connections

`28 × 28 × 6 × 26`

`= 4,704 × 26`

`=` `122,304`

WHY 32×32?

MNIST digits are 28×28. Padding to 32×32 lets stroke ends and corners near the border fall in the **centre** of the top-level receptive fields.

WEIGHT SHARING IN ACTION

Each of the 4,704 output units reuses the same 26 numbers of its filter: 156 parameters but 122,304 connections, a **reuse factor of 784** (= 28 × 28 positions).

Step 2 · S2: trainable subsampling, not max pooling

Input 28×28×6 · window 2×2 · S = 2

Output size

`O = (28 − 2)/2 + 1 = 14`

→ `14×14×6`

Parameters

`1 coefficient + 1 bias per map`

`2 × 6 =` `12`

Connections

`14 × 14 × 6 × (4 + 1)`

`= 1,176 × 5 =` `5,880`

WHAT ONE S2 UNIT COMPUTES

$$y = f(w \cdot (x1+x2+x3+x4) + b)$$

`w, b`: learned, one pair per map · `f`: squashing function. It sums the 2×2 window, scales, shifts and squashes. With w = 0.25 it is an average pool plus a learned gain.

Subsampling reduces sensitivity to small shifts and distortions. Because the coefficient is learned, S2 can act anywhere from near-linear (small w) to near-binary (large w, saturating the squash).

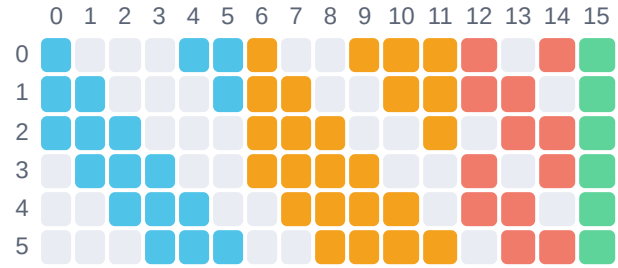
Modern versions

Replace this with **max pooling**: no parameters, no activation. Worked side by side in Part B.

Step 3 · C3: sparse connections between maps

Input 14×14×6 · K = 16 · F = 5. Each C3 map is connected to only **some** of the six S2 maps.

WHICH S2 MAPS (ROWS) FEED EACH C3 MAP (COLUMNS)



0–5: 3 contiguous 6–11: 4 contiguous

12–14: 4 non-contiguous 15: all 6

60 of the 96 possible map pairs are connected.

Output size

O = (14 − 5)/1 + 1 = 10 → `10×10×16`

Parameters

```
6 × (3×25 + 1) = 456
6 × (4×25 + 1) = 606
3 × (4×25 + 1) = 303
1 × (6×25 + 1) = 151
```

total = `1,516`

Connections

10×10 × 1,516 = `151,600`

WHY SPARSE? TWO REASONS FROM THE PAPER

(1) It keeps the number of connections within bounds for 1990s hardware. (2) It **breaks symmetry**: different C3 maps see different input combinations, so they are forced to learn different features. A fully connected C3 would need 2,416 parameters (see Part B). The same idea returns in AlexNet as grouped convolution.

Steps 4–5 · S4 and C5: a convolution that is secretly FC

S4 subsampling — 10×10×16 · 2×2 · S = 2

O = (10 − 2)/2 + 1 = 5 → `5×5×16`

```
Params = 2 × 16 = 32
Connections = 5×5×16 × 5 = 2,000
```

C5 convolution — 5×5×16 · K = 120 · F = 5

O = (5 − 5)/1 + 1 = 1 → `1×1×120`

```
Params = (5×5×16 + 1) × 120
= 401 × 120 = 48,120
Connections = 48,120
```

WHY CALL C5 A CONVOLUTION?

On a 32×32 input its 5×5 filter covers the whole 5×5 map, so it equals a fully connected layer (400 → 120); it has the same parameter count as `nn.Linear(400, 120)`: 400 × 120 + 120 = 48,120. On a larger input it would slide and produce a map, so the paper kept the conv label. This is the first appearance of a **"fully convolutional"** idea: a network with no true FC layers can run on any input size.

Steps 6–7 · F6 and the RBF output layer

F6 fully connected, 120 → 84

```
Params = (120 + 1) × 84
= 121 × 84 = 10,164
Connections = 10,164
```

ACTIVATION (SCALED TANH)

$$f(a) = 1.7159 \cdot \tanh(2a/3)$$

Chosen so that f(1) = 1 and f(−1) = −1. The same squashing function is used throughout the network.

Output: 10 Euclidean RBF units

$$y_i = \Sigma_j (x_j - w_{ij})^2,\ j = 1 \ldots 84$$

Output = distance to a class prototype: **smallest wins**

Weights: 84 × 10 = 840
fixed at +1 / −1, not trained

WHY 84?

Each prototype is a **7×12 bitmap** of the character drawn in ±1 pixels: 7 × 12 = 84.

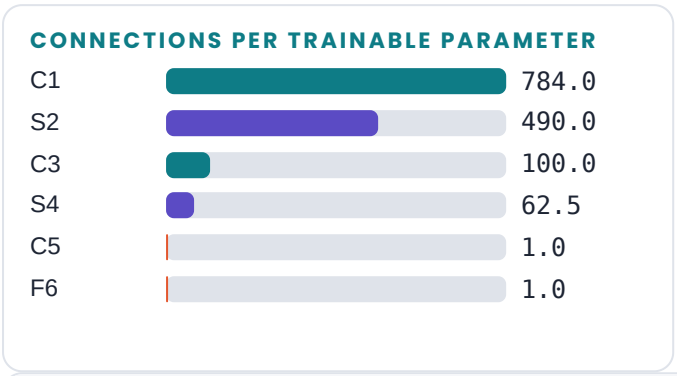
The RBF prototypes were designed by hand as stylised 7×12 bitmaps of ASCII characters, so confusable characters (O, 0, o) get similar codes and a linguistic post-processor can fix them. Total connections including this layer: 340,068 + 840 = 340,908. Trainable parameters exclude these 840 fixed weights.

Step 8 · The full tally, original LeNet-5

Layer	Output shape	Trainable params	Connections
Input	32×32×1	–	–
C1 conv 5×5, 6	28×28×6	156	122,304
S2 subsample 2×2	14×14×6	12	5,880
C3 conv 5×5, 16 (sparse)	10×10×16	1,516	151,600
S4 subsample 2×2	5×5×16	32	2,000
C5 conv 5×5, 120	1×1×120	48,120	48,120
F6 full	84	10,164	10,164
Output RBF (fixed)	10	0	840
Total		**60,000**	**340,908**

97% 156 + 12 + 1,516 + 32 + 48,120 + 10,164 = 60,000. **C5 and F6 alone hold 97% of the parameters** (58,284 of 60,000 = 97.1%). The same pattern appears in AlexNet, where FC layers held 96%.

03 Weight sharing: each parameter is reused many times



784×
C1: 122,304 connections from 156 params

1×
C5 and F6: no sharing, every weight used once

≈ 5.7×
whole network: 340,908 / 60,000

Reuse factor = number of output positions

```
C1: 28×28 = 784
S2: 14×14 = 196 positions × 5 connections per unit / 2 params per map = 490
C3: 10×10 = 100
S4: 5×5×5 / 2 = 62.5
```

THE LESSON

Early layers are compute-heavy and parameter-light; late layers are the reverse. Same lesson as the AlexNet deck.

SLIDES 12–19 · PART B

04 Modern LeNet-5 and what changed

The version in today's textbooks and PyTorch tutorials keeps the shape of LeNet-5 but swaps in newer components. Each change is worked by hand.

KNOW BOTH VERSIONS

"LeNet-5" in most courses today means the **modern** variant, but exam questions often ask about the **original** paper.

B1 · Side by side: original (1998) vs modern

Aspect	Original LeNet-5 (1998)	Modern LeNet-5
Input	32×32 (MNIST padded)	28×28 with padding 2 in conv 1
Activation	Scaled tanh 1.7159·tanh(2a/3)	ReLU
Pooling	Trainable subsampling (w, b, squash)	Max pooling, no parameters
C3 connectivity	Sparse table, 60 of 96 map pairs	Full: every map sees all 6
C5	Conv 5×5 on a 5×5 map	Flatten + Linear(400, 120)
Output	10 RBF units, fixed ±1 weights	Linear(84, 10) + softmax
Loss	MSE-style on RBF distances	Cross-entropy
Optimiser	Stochastic diagonal Levenberg–Marquardt	SGD with momentum or Adam
Trainable parameters	**60,000**	**61,706**

WHAT DID NOT CHANGE

5×5 filters, 6 then 16 feature maps, 2×2 downsampling, 120 and 84 hidden units. The architecture shape survives; only the components were modernised.

B2 · Modern LeNet-5, layer by layer

Layer	Output	Params	MACs
Conv 5×5, 6, P = 2	28×28×6	156	117,600
MaxPool 2×2	14×14×6	0	–
Conv 5×5, 16	10×10×16	2,416	240,000
MaxPool 2×2	5×5×16	0	–
Flatten	400	0	–
FC 400 → 120	120	48,120	48,000
FC 120 → 84	84	10,164	10,080
FC 84 → 10	10	850	840
Total		**61,706**	**416,520**

61,706
parameters

0.42 M
multiply-accumulates

≈ 1,740×
fewer MACs than AlexNet (724 M)

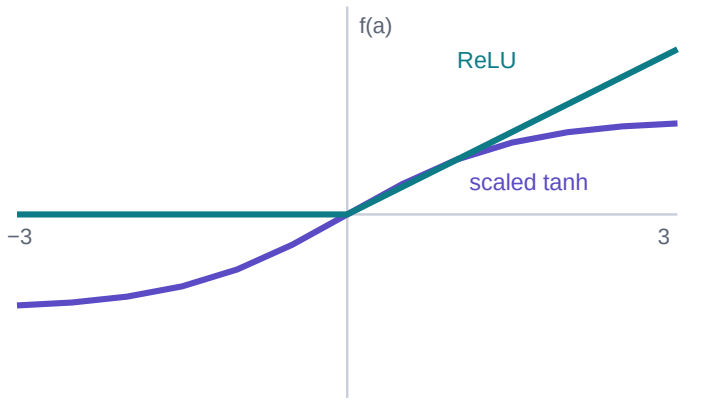
The working

```
Conv 1 size: (28 − 5 + 4)/1 + 1 = 28
Conv 2 params: (5×5×6 + 1) × 16 = 2,416
MACs = outputs × F×F×C_in
conv1 = 28×28×6 × 25 = 117,600 conv2 = 10×10×16 × 150 = 240,000
FC = 48,000 + 10,080 + 840 = 58,920 Total 416,520
AlexNet 724,406,816 / 416,520 ≈ 1,739
```

The whole modern LeNet-5 needs less compute than a single AlexNet output channel map in Conv1.

B3 · Scaled tanh vs ReLU, by hand

F(A) FOR A FROM -3 TO 3



a	−3	−1	0	0.5	1	2	3
tanh	−1.654	−1.0	0	0.552	1.0	1.493	1.654
ReLU	0	0	0	0.5	1	2	3

f(a) = 1.7159 · tanh(2a/3)

```
a = 0.5: 1.7159 × tanh(0.333)
= 1.7159 × 0.3215 = 0.552
a = 1 : 1.7159 × 0.5828 = 1.000
a = 3 : 1.7159 × 0.9640 = 1.654
```

Gradient

```
f'(a) = 1.1439 · (1 − tanh²(2a/3))
f'(0) = 1.1439
f'(3) = 1.1439 × (1 − 0.9294)
= 0.081 (14× smaller)

ReLU: f'(a) = 1 for every a > 0
```

WHY IT MATTERS

1.1439 = 1.7159 × 2/3, and tanh(2)² = 0.9294. The flattening at large |a| (**saturation**) shrinks the gradient, which is why deep tanh networks train slowly (**vanishing gradients**). LeNet-5 is shallow enough that tanh worked; AlexNet showed ReLU trains about 6× faster.

B4 · Trainable subsampling vs max pooling on one 4×4 map

FEATURE MAP (4×4)

2	**4**	1	0
1	**3**	2	2
0	1	**5**	**1**
2	1	**0**	**3**

2×2 windows, stride 2 → 2×2 output. Two of the four windows highlighted.

Original: y = f(w·Σ + b)

```
w = 0.25, b = 0.1

TL Σ = 2+4+1+3 = 10
0.25×10 + 0.1 = 2.60
f(2.60) = 1.612
TR Σ = 5 → 1.35 → 1.229
BL Σ = 4 → 1.10 → 1.073
BR Σ = 9 → 2.35 → 1.573
```

1.61	1.23
1.07	1.57

2 learned numbers per map

Modern: max

```
TL max = 4
TR max = 2
BL max = 2
BR max = 5
```

4	2
2	5

No parameters.
Keeps the strongest response.

WHY MAX WON

f is the scaled tanh: f(2.60) = 1.7159 × tanh(1.733) = 1.7159 × 0.9395 = 1.612. Averaging dilutes a single strong activation (the 5 in BR contributes only 1.25 after scaling), while max keeps it. Empirically, max pooling works better for recognition, and it removes parameters.

B5 · Sparse (1998) vs full (modern) C3

Original: sparse table

```
Params = 456 + 606 + 303 + 151
= 1,516

Connections = 100 × 1,516
= 151,600
```

Modern: every map sees all 6

```
Params = (5×5×6 + 1) × 16
= 151 × 16 = 2,416

Connections = 100 × 2,416
= 241,600
```

+900
extra parameters for full C3

+59%
more connections (compute): 241,600 / 151,600 = 1.59

On 1998 hardware that extra compute mattered; on a modern GPU it is negligible, and full connectivity is simpler. Symmetry breaking is now handled by **random initialisation**. The idea returns as grouped conv in AlexNet and ResNeXt.

B6 · RBF distance vs softmax probability

Original: RBF (4-D toy version)

```
x = [0.9, −0.8, 0.7, −1.0]
w_A = [ 1, −1, 1, −1 ]
w_B = [−1, 1, 1, −1 ]

y_A = 0.1² + 0.2² + 0.3² + 0²
= 0.01 + 0.04 + 0.09 = 0.14
y_B = 1.9² + 1.8² + 0.3² + 0²
= 3.61 + 3.24 + 0.09 = 6.94

Smallest distance → class A
```

Modern: Linear + softmax

```
logits z = [2.1, −0.4]

e^2.1 = 8.166
e^−0.4 = 0.670
Sum = 8.836

p_A = 8.166 / 8.836 = 0.924
p_B = 0.670 / 8.836 = 0.076

Largest probability → class A
Loss = −ln(0.924) = 0.079
```

KEY CONTRAST

RBF gives a distance: smaller is better, fixed prototypes, not a probability. **Softmax** gives a probability distribution: larger is better, learned weights, and it pairs naturally with cross-entropy loss. Real LeNet-5 uses 84-D vectors and 10 fixed prototypes.

B7 · Modern LeNet-5 in PyTorch

```
class LeNet5(nn.Module):
  def __init__(self):
    super().__init__()
    self.features = nn.Sequential(
      nn.Conv2d(1, 6, 5, padding=2),
nn.ReLU(),
      nn.MaxPool2d(2),
      nn.Conv2d(6, 16, 5), nn.ReLU(),
      nn.MaxPool2d(2))
    self.classifier = nn.Sequential(
      nn.Flatten(),
      nn.Linear(16*5*5, 120), nn.ReLU(),
      nn.Linear(120, 84), nn.ReLU(),
      nn.Linear(84, 10))
  def forward(self, x):
    return self.classifier(self.features(x))
```

Check the count

```
sum(p.numel() for p
in model.parameters())
# 61706
```

TRAIN IT

`nn.CrossEntropyLoss` + Adam on MNIST reaches about **99%** test accuracy in a few epochs.

NO SOFTMAX IN THE MODEL

`CrossEntropyLoss` applies log-softmax internally, so the last layer outputs raw logits.

Lab extension

Swap ReLU for Tanh and MaxPool2d for AvgPool2d to get closer to the original, and compare training curves.

SLIDE 20 · WRAP-UP

05 What to take away

Same skeleton

conv 5×5 → pool → conv 5×5 → pool → 120 → 84 → 10 survives unchanged from 1998 to today.

Modern parts

ReLU, max pooling, full C3, softmax with cross-entropy. Parameters: 60,000 → 61,706.

Sharing is the point

C1 reuses each weight 784 times. FC layers hold 97% of parameters but little compute.

Try it: modern LeNet-5 on CIFAR-10 (from the slide, answers from the speaker notes)

Input 32×32×3, conv 1 has **no padding**, full C3, 10 classes. (a) Output shape after each layer? (b) Parameters in conv 1 and in the whole network? (c) Which layer's parameter count changed, and why?

(a) Shapes

```
32×32×3
→ conv 5×5, 6: 28×28×6
→ pool: 14×14×6
→ conv 5×5, 16: 10×10×16
→ pool: 5×5×16
→ 400 → 120 → 84 → 10
```

(b) Parameters

```
Conv 1 = (5×5×3 + 1) × 6
= 456

Total = 456 + 2,416 +
48,120
+ 10,164 + 850
= 62,006
```

(c) WHAT CHANGED

Only conv 1 changes (**156 → 456**), because filter depth must equal the 3 input channels. Every later layer sees the same shapes as before.

Must-know numbers

Layer	Original: shape · params · connections	Modern: params
C1	28×28×6 · 156 · 122,304	156
S2	14×14×6 · 12 · 5,880	0 (max pool)
C3	10×10×16 · 1,516 · 151,600 (sparse)	2,416 (full)
S4	5×5×16 · 32 · 2,000	0 (max pool)
C5	1×1×120 · 48,120 · 48,120	48,120 (Linear 400→120)
F6	84 · 10,164 · 10,164	10,164
Output	RBF, 840 fixed weights · 0 · 840	850 (Linear 84→10)
Total	**60,000 params · 340,908 connections**	**61,706 params · 416,520 MACs**

Test yourself

Q1. Why does C1 have 156 parameters but 122,304 connections?

Q2. How many trainable parameters does one S2 map have, and what are they?

Q3. Give the two reasons for C3's sparse connection table.

Q4. Why is C5 called a convolution if it acts like an FC layer?

Q5. Why are the RBF output weights not counted in the 60,000?

A1. Weight sharing: the same 26 numbers are reused at all 28 × 28 = 784 positions of each of 6 maps.

A2. Two: one coefficient w and one bias b.

A3. Fewer connections for 1990s hardware, and breaking symmetry so maps learn different features.

A4. On a 32×32 input it covers the whole 5×5 map; on a larger input it would slide and produce a map.

A5. They are fixed at ±1 (hand-designed 7×12 prototypes), not trained.